Tuple

Basic Theory:

A tuple is a sequence of immutable Python objects. Tuples are sequences, just like lists. The differences between tuples and lists are, the tuples cannot be changed unlike lists and tuples use parentheses, whereas lists use square brackets.

Creating a tuple is as simple as putting different comma-separated values. Optionally you can put these comma-separated values between parentheses also. For example –

```
tup1 = ('physics', 'chemistry', 1997, 2000);
tup2 = (1, 2, 3, 4, 5 );
tup3 = "a", "b", "c", "d";
```

The empty tuple is written as two parentheses containing nothing –

```
tup1 = ();
```

To write a tuple containing a single value you have to include a comma, even though there is only one value –

```
tup1 = (50,);
```

Like string indices, tuple indices start at 0, and they can be sliced, concatenated, and so on.

1. Do the following in Python Shell.

a) Save a tuple. Print it. Delete it.

```
>>> tup=(1,2,3)
>>> print(tup)
(1, 2, 3)
>>> del(tup)
>>> print(tup)
```

b) Save two tuples. Concatenate them. Print it without using loops.

```
>>> tup=(1,2,3)
>>> next=(10,20,30)
>>> new=tup+next
>>> print(new)
(1, 2, 3, 10, 20, 30)
```

c) Suppose a tuple contains one item. Now store the same item for five times without using loop.

```
>>> tup=(5,)
>>> tup=tup*5
>>> print(tup)
(5, 5, 5, 5, 5)
```

d) Save a tuple. Print i^{th} to j^{th} item without using loop.

```
>>> tup=(1,2,3,10,20,30)
>>> print(tup[2:4])
(3, 10)
```

e) Convert a list in a tuple.

```
>>> tup=(1,2,3)
>>> new=list(tup)
>>> print(new)
```

[1, 2, 3]

f) Find maximum and minimum item in a tuple. Find the length of a tuple.

```
>>> tup=(1,2,3,10,20,30)
>>> max(tup)
30
>>> min(tup)
1
>>> len(tup)
6
```

2. Write a Python program to reverse a tuple.

```
#create a tuple
x = ("debashis")
# Reversed the tuple
y = reversed(x)
print(tuple(y))
#create another tuple
x = (5, 10, 15, 20)
# Reversed the tuple
y = reversed(x)
print(tuple(y))
```

4. Write a Python program to find the index of an item of a tuple. Convert a string to a tuple. Check it for all possible parameters of index function. Check it for an item which is not present.

```
#create a tuple
tuplex = tuple("index tuple")
print(tuplex)
#get index of the first item whose value is passed as parameter
index = tuplex.index("p")
print(index)
#define the index from which you want to search
index = tuplex.index("p", 5)
print(index)
#define the segment of the tuple to be searched
index = tuplex.index("e", 3, 6)
print(index)
#if item not exists in the tuple return ValueError Exception
index = tuplex.index("y")
```

5. Write a program in Python to do slicing in all possible ways with all possible parameters, providing positive and negative values for step. Also, perform slicing from start and end both.

```
#create a tuple
tuplex = (2, 4, 3, 5, 4, 6, 7, 8, 6, 1)
#used tuple[start:stop] the start index is inclusive and the stop index
slice = tuplex[3:5]
#is exclusive
print(slice)
#if the start index isn't defined, is taken from the beg inning of the tuple
```

```
slice = tuplex[:6]
print(slice)
#if the end index isn't defined, is taken until the end of the tuple
slice = tuplex[5:]
print(slice)
#if neither is defined, returns the full tuple
slice = tuplex[:]
print(slice)
#The indexes can be defined with negative values
slice = tuplex[-8:-4]
print(slice)
#create another tuple
tuplex = tuple("HELLO WORLD")
print(tuplex)
#step specify an increment between the elements to cut of the tuple
#tuple[start:stop:step]
slice = tuplex[2:9:2]
print(slice)
#returns a tuple with a jump every 3 items
slice = tuplex[::4]
print(slice)
#when step is negative the jump is made back
slice = tuplex[9:2:-4]
print(slice)
```

Dictionaries in Python

Basic Theory:

Each key is separated from its value by a colon (:), the items are separated by commas, and the whole thing is enclosed in curly braces. An empty dictionary without any items is written with just two curly braces, like this: {}.

Keys are unique within a dictionary while values may not be. The values of a dictionary can be of any type, but the keys must be of an immutable data type such as strings, numbers, or tuples.

Accessing Values in Dictionary

```
dict = {'Name': 'Zara', 'Age': 7, 'Class': 'First'}
print "dict['Name']: ", dict['Name']
print "dict['Age']: ", dict['Age']
```

Updating Dictionary

```
dict = {'Name': 'Zara', 'Age': 7, 'Class': 'First'}
dict['Age'] = 8; # update existing entry
dict['School'] = "DPS School"; # Add new entry
print "dict['Age']: ", dict['Age']
print "dict['School']: ", dict['School']
```

1. Do the following in Python shell.

a) Create a dictionary which has at least 5 items. Print it.

```
>>> num={1:'one',2:'two',3:'three',4:'four',5:'five'}
>>> print(num)
```

{1: 'one', 2: 'two', 3: 'three', 4: 'four', 5: 'five'}

b) Update any one of the item. Print it.

```
>>> num={1:'one',2:'two',3:'three',4:'four',5:'five'}
>>> print(num)
{1: 'one', 2: 'two', 3: 'three', 4: 'four', 5: 'five'}
>>> d={2:"Two"}
>>> num.update(d)
>>> print(num)
{1: 'one', 2: 'Two', 3: 'three', 4: 'four', 5: 'five'}
```

c) Add one more item. Print it.

```
>>> num={1:'one',2:'two',3:'three',4:'four',5:'five'}
>>> print(num)
{1: 'one', 2: 'two', 3: 'three', 4: 'four', 5: 'five'}
>>> d1={6:"Six"}
>>> num.update(d1)
>>> print(num)
{1: 'one', 2: 'two', 3: 'three', 4: 'four', 5: 'five', 6: 'Six'}
```

d) Delete a particular item mentioning key value. Print it.

```
>>>num={1: 'one', 2: 'two', 3: 'three', 4: 'four', 5: 'five', 6: 'Six'}
>>>print(num)
{1: 'one', 2: 'two', 3: 'three', 4: 'four', 5: 'five', 6: 'Six'}
>>> del num[6]
>>> print(num)
{1: 'one', 2: 'two', 3: 'three', 4: 'four', 5: 'five'}
```

e) Delete the first item without mentioning key value. Print the dictionary.

```
>>>person = {'name': 'Phill', 'age': 22, 'salary': 3500.0}
>>>result = person.popitem()
>>>print('person = ',person)
person =  {'name': 'Phill', 'age': 22}
>>>print('Return Value = ',result)
Return Value =  ('salary', 3500.0)
```

f) Remove all items from the dictionary.

```
>>> num={1:'one',2:'two',3:'three',4:'four',5:'five'}
>>> num.clear()
>>> print(num)
{}
```

g) Delete the dictionary.

```
>>> num={1:'one',2:'two',3:'three',4:'four',5:'five'}
>>> del num
>>> print(num)
Traceback (most recent call last):
  File "<pyshell#17>", line 1, in <module>
    print(num)
NameError: name 'num' is not defined
```

h) Create a dictionary with each item being a pair of a number and its square. Do it in single line of code.

```
>>> d={i:i**3 for i in range(1,10)}
>>> print(d)
```

{1: 1, 2: 8, 3: 27, 4: 64, 5: 125, 6: 216, 7: 343, 8: 512, 9: 729}

i) Do the same as above only for odd numbers.

```
>>> d={i:i**3 for i in range(1,10,2)}
>>> print(d)
{1: 1, 3: 27, 5: 125, 7: 343, 9: 729}
```

h) Print all items and keys of the dictionary.

```
>>> d = {'a': 100, 'b':200, 'c':300}
>>> d.keys()
dict  keys(['a', 'b', 'c'])
>>> d.items()
dict_items([('a', 100), ('b', 200), ('c', 300)])
```

j) Calculate the length of the dictionary.

```
>>> d = {'a': 100, 'b':200, 'c':300}
>>> len(d)
3
```

k) Sort the items in the dictionary.

```
>>> pyDict = {'e': 1, 'a': 2, 'u': 3, 'o': 4, 'i': 5}
>>> print(sorted(pyDict, reverse=True))
['u', 'o', 'i', 'e', 'a']
```

l) Create two dictionaries. Concatenate them.

```
>>> d1={1:100,2:200,3:300}
>>> d2={4:400,5:500}
>>> d1.update(d2)
>>> print(d1)
{1: 100, 2: 200, 3: 300, 4: 400, 5: 500}
```

m) Pop an element not present from the dictionary, provided a default value.

```
>>>sales = { 'apple': 2, 'orange': 3, 'grapes': 4 }
>>>element = sales.pop('guava', 'banana')
>>>print('The popped element is:', element)
The popped element is: banana
>>>print('The dictionary is:', sales)
The dictionary is: {'grapes': 4, 'orange': 3, 'apple': 2}
```

n) Sort the items in frozen set.

```
>>> pyFSet = frozenset(('e', 'a', 'u', 'o', 'i'))
>>> print(sorted(pyFSet, reverse=True))
['u', 'o', 'i', 'e', 'a']
```

2. Write a program to check whether an item is present or not.

```
dict = {'a': 100, 'b':200, 'c':300}
key=input(“Enter a key:”)
if dict.has_key(key):
   print "Present, value =", dict[key]
else:
   print "Not present"
```

3. Write a program to print all the items of the dictionary using loop.

```
statesAndCapitals = {
                    'Gujarat' : 'Gandhinagar',
                    'Maharashtra' : 'Mumbai',
                    'Rajasthan' : 'Jaipur',
                    'Bihar' : 'Patna'
                   }
print('List Of given states:\n')
  # Iterating over keys
for state in statesAndCapitals:
    print(state)
```

4. Write a program to map two lists (one containing color names and the other containing color codes) into dictionary.

```
keys = ['red', 'green', 'blue']
values = ['#FF0000','#008000', '#0000FF']
color  dictionary = dict(zip(keys, values))
print(color_dictionary)
```

Output:

```
{'green': '#008000', 'blue': '#0000FF', 'red': '#FF0000'}
```

5. Write a program to get the maximum and minimum value in a dictionary.

```
my_dict = {'x':500, 'y':5874, 'z': 560}

key_max = max(my_dict.keys(), key=(lambda k: my_dict[k]))
key_min = min(my_dict.keys(), key=(lambda k: my_dict[k]))

print('Maximum Value: ',my  dict[key  max])
print('Minimum Value: ',my_dict[key_min])
```

Output:

```
Maximum Value:  5874
Minimum Value:  500
```

Assignments:

1. Write a program in Python to do searching either linear or binary. The choice will be provided by the user.
2. Write a Python program to count the elements in a list until an element is a tuple.
3. . Write a program to store student data in dictionary (name, class, subjects). Remove duplicate entries.
4. Write a Python program to sum all the items in a dictionary.
5. Write a Python program which creates two dictionaries. One that stores conversion values from meters to centimeters and the other that stores the reverse.
6. Write a Python program that inverts a dictionary, i.e., it makes key of one dictionary value of another and vice versa.
7.